operation on your database. Imagine an attacker searches your website using the text

```
'; DROP TABLE Recipes; --
```

Your app assigns this to the `search` parameter, and the SQL query executed against your database becomes

```
SELECT * FROM Recipes WHERE Name = ''; DROP TABLE Recipes; --'
```

Simply by entering text into the search form of your app, the attacker has deleted the entire Recipes table from your app! That's catastrophic, but an SQL injection vulnerability provides more or less unfettered access to your database. Even if you've set up database permissions correctly to prevent this sort of destructive action, attackers will likely be able to read all the data from your database, including your users' details.

The simple way to prevent this from happening is to avoid creating SQL queries by hand this way. If you do need to write your own SQL queries, don't use string concatenation, as in listing 29.6. Instead, use parameterized queries, in which the (potentially unsafe) input data is separate from the query itself, as shown here.

Listing 29.7 Avoiding SQL injection by using parameterization

```
public IList<User> FindRecipe(string search)
{
    return _context.Recipes
        .FromSqlRaw( "SELECT * FROM Recipes WHERE Name = '{0}'",  ❶
                search)                                           ❷
        .ToList();
}
```

❶ The SQL query uses a placeholder {0} for the parameter.

❷ The dangerous input is passed as a parameter, separate from the query.